R3.05 · TD-TP 04

PROGRAMMATION SYSTÈME

L'allocation dynamique

BUT2 Informatique · R3.05 · 2026-2027 · Séance de 2 h

Le tas du LAMAE est hanté. Des blocs de mémoire jamais rendus, des pointeurs vers des pièces démolies, des cases de trop... Ta mission : apprendre à réserver et à rendre la mémoire proprement, chasser les fantômes, puis construire le banc d'essai des tris du labo.

OBJECTIFS DE LA SÉANCE

- ☐ Réserver de la mémoire sur le **tas** avec `malloc`, `calloc` et `realloc`, et la rendre avec `free`.
- ☐ Distinguer un pointeur vers une variable existante d'un pointeur vers un bloc alloué.
- ☐ Allouer des tableaux à une et deux dimensions dont la taille est connue à l'exécution.
- ☐ Situer les données d'un processus : code, données, tas, pile.
- ☐ Détecter fuites, use-after-free et double free avec `valgrind` et `-fsanitize=address`.
- ☐ Réutiliser les pointeurs de fonctions (sujet 03) dans un système de tri modulaire.

FIL ROUGE **Opération « Manoir du tas »**

Deux actes. **Acte I** : tu explores le manoir (le tas), tu y bâtis des tableaux, puis tu chasses les quatre fantômes cachés dans `fantomes.c`. **Acte II** : tu construis le **banc d'essai des tris** du LAMAE, qui alloue, copie, trie et compare. Chaque mission rapporte un drapeau `FLAG{...}` à noter dans `badge04.txt`.

Mission	Contenu	Durée
1 · Le premier bloc	`malloc`, `sizeof`, `NULL`, `free` (Ex 1, 2)	20 min
2 · Les plans du manoir	Tableau 2D dynamique, segments mémoire (Ex 3, 4)	20 min
3 · La chasse aux fantômes	valgrind, ASan : fuite, use-after-free, double free, débordement (Ex 5)	20 min
4 · L'extension	`realloc` sans perdre le bloc (Ex 6)	15 min
5 · Le banc d'essai	Tris avec statistiques, utilitaires, menu (Ex 7 à 10)	45 min
Bonus ★★★	Tri rapide itératif, `qsort`, grands tableaux (Ex 11)	autonomie

COMPILATION DE LA SÉANCE

On compile avec les options habituelles ; pour la chasse aux fantômes, on ajoute l'instrumentation d'AddressSanitizer.

TERMINAL

```
$ gcc -std=c17 -Wall -Wextra -pedantic -g prog.c -o prog
$ gcc -std=c17 -Wall -Wextra -pedantic -g -fsanitize=address prog.c -o prog_asan
$ valgrind --leak-check=full ./prog
```

Mission 1 — Le premier bloc

Pourquoi allouer dynamiquement ?

Avant de manipuler un pointeur, et notamment de lui appliquer l'opérateur `*`, il faut l'**initialiser**. On peut lui affecter l'adresse d'une variable existante (`p = &i;`, sujet 03). On peut aussi lui faire désigner un **nouvel espace mémoire**, réservé à l'exécution : c'est l'**allocation dynamique**. Elle est indispensable quand la taille des données n'est connue qu'à l'exécution (nombre d'éléments saisi par l'utilisateur, taille d'un fichier...).

> **PIÈGE CLASSIQUE** **Un pointeur local n'est PAS initialisé à NULL**
>
> Une variable locale non initialisée a une valeur **indéterminée** : un pointeur déclaré par `int *p;` dans une fonction contient n'importe quelle adresse. Seules les variables globales et `static` sont mises à zéro (`NULL` pour un pointeur). Prenez l'habitude d'écrire `int *p = NULL;` : un déréférencement de `NULL` plante immédiatement (Segmentation fault), alors qu'une adresse au hasard peut corrompre la mémoire en silence. Le test `p == NULL` permet de savoir si un pointeur désigne un objet.

La fonction malloc

`malloc` (de `<stdlib.h>`) réserve un bloc de `nombre_octets` octets sur le **tas** et renvoie son adresse, de type `void *` (pointeur générique), ou `NULL` si l'allocation échoue :

```c
void *malloc(size_t nombre_octets);
```

En C, un `void *` se convertit implicitement vers n'importe quel type de pointeur : **le cast est inutile** (il était nécessaire en C pré-ANSI, où `malloc` renvoyait un `char *`). On calcule la taille avec `sizeof`, qui rend le code portable :

```c
#include <stdlib.h>

int *p = malloc(sizeof *p);          /* forme recommandée : suit le type de p */
int *q = malloc(sizeof(int));        /* équivalent */
int *r = malloc(4);                  /* À ÉVITER : un int ne fait pas 4 octets partout */
```

Le programme suivant réserve un `int` sur le tas et y range la valeur de `i` :

avant_apres.c

```c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int i = 3;
    int *p = NULL;

    printf("valeur de p avant allocation = %p\n", (void *)p);
    p = malloc(sizeof *p);
    if (p == NULL) {                    /* TOUJOURS tester */
        perror("malloc");
        return EXIT_FAILURE;
    }
    printf("valeur de p après allocation = %p\n", (void *)p);
    *p = i;
    printf("valeur de *p = %d\n", *p);
    free(p);                            /* on rend le bloc */
    p = NULL;
    return EXIT_SUCCESS;
}
```

Avant l'allocation, `*p` n'a aucun sens. Après, on se trouve dans la configuration suivante (adresses **fictives** : les vraies varient à chaque exécution, à cause de l'ASLR) :

Objet	Adresse	Valeur
`i`	`0x7ffc1000` (pile)	3
`p`	`0x7ffc1008` (pile)	`0x5555a2c0`
`*p`	`0x5555a2c0` (tas)	? (int non initialisé), puis 3

Comparez avec `int *p = &i;` : là, `i` et `*p` sont **le même objet** (même adresse), toute modification de l'un modifie l'autre, et aucune allocation n'est nécessaire puisque l'espace de `i` est déjà réservé. Dans le programme ci-dessus, `*p` et `i` ont la même valeur mais des adresses différentes.

Plusieurs objets contigus, calloc et free

`malloc` peut réserver de la place pour plusieurs objets contigus : `p = malloc(2 * sizeof *p);` réserve deux `int`, accessibles par `*p` et `*(p + 1)`, c'est-à-dire `p[0]` et `p[1]`.

`calloc(nb_objets, taille_objet)` fait la même chose **et met tous les octets à zéro** :

C

```c
int *t = calloc(N, sizeof *t);      /* N int valant 0 */
/* équivaut à : */
int *u = malloc(N * sizeof *u);
for (size_t k = 0; k < N; k++)
    u[k] = 0;
```

`calloc` a un autre avantage : il **vérifie** que `N * taille` ne déborde pas (voir l'encadré sécurité de l'exercice 2). Il n'est pas forcément « plus rapide » : il l'est souvent pour de gros blocs, que le système fournit déjà remplis de zéros.

Quand on n'a plus besoin d'un bloc, on le rend avec `free(p);`. À chaque `malloc`, `calloc` ou `realloc` réussi doit correspondre **exactement un** `free`. `free(NULL)` est autorisé et ne fait rien.

EX 1 Avant, après

★ 5 min

Compilez et exécutez `avant_apres.c`. Ajoutez une allocation de **deux** `int` (`i = 3`, `j = 6`) et affichez `p`, `*p`, `p + 1` et `*(p + 1)` avec `%p` et `%d`. De combien d'octets `p + 1` est-il décalé ? **Drapeau :** `FLAG{` + le nombre d'octets réservés par `malloc(2 * sizeof(int))` sur votre machine + `}`.

EX 2 Le tableau à la demande

★ 15 min

Écrivez un programme qui demande à l'utilisateur un nombre d'éléments `n`, alloue dynamiquement un tableau de `n` entiers, le remplit (par exemple avec `i * i`), l'affiche avec les adresses de chaque case, puis libère la mémoire. Testez avec `n = 5`, `n = 0`, `n = -1` et `n = abc`.

> **SÉCURITÉ** **Calcul de taille : le dépassement silencieux**
>
> `malloc(n * sizeof(int))` : si `n` est énorme (saisi par l'utilisateur ou lu dans un fichier), le produit `n * sizeof(int)` peut **dépasser** la capacité de `size_t` et revenir à une petite valeur. `malloc` réserve alors un minuscule bloc… que le programme remplit avec `n` éléments : débordement du tas. Avant d'allouer, testez `if (n > SIZE_MAX / sizeof(int))` (`<stdint.h>`), ou utilisez `calloc(n, sizeof(int))` qui fait ce test pour vous. Le guide ANSSI demande de vérifier tout calcul de taille.

Mission 2 — Les plans du manoir

EX 3 Une grille de caractères

★★ 10 min

Allouez dynamiquement un tableau de caractères à **deux dimensions** de `L` lignes et `C` colonnes : un tableau de `L` pointeurs (`char **grille`), puis une ligne de `C + 1` caractères pour chacun (le `+ 1` pour le `'\0'` qui permet d'afficher la ligne avec `%s`). Remplissez-le (par exemple `'A' + (i + j) % 26`), affichez-le, puis libérez **toutes** les lignes, **puis** le tableau de pointeurs.

> **INDICE** **Écrire deux fonctions**
>
> `char **creerGrille(size_t nbLignes, size_t nbColonnes)` et `void libererGrille(char **g, size_t nbLignes)`. Si l'allocation de la ligne 3 échoue, il faut libérer les lignes 0 à 2 déjà allouées : allouer le tableau de pointeurs avec `calloc` (toutes les lignes à `NULL`) permet d'appeler simplement `libererGrille`, puisque `free(NULL)` ne fait rien.

> **PIÈGE CLASSIQUE** **L'ordre de libération**
>
> `free(grille)` avant `free(grille[i])` : vous perdez les adresses des lignes (fuite), et lire `grille[i]` après `free(grille)` est un use-after-free. On libère de l'intérieur vers l'extérieur.

EX 4 Statique ou dynamique : où vivent les données ?

★★ 10 min

Dans le même programme, déclarez un tableau 2D **global** `int global[3][4]`, un tableau 2D **local** `int local[3][4]` et un tableau 2D **dynamique** `int **dyn` (3 lignes de 4 `int`). Affichez avec `%p` l'adresse de `main`, puis celles de `global`, `local`, `&dyn`, `dyn`, de chaque `local[i]` et `dyn[i]` et de quelques `local[i][j]` et `dyn[i][j]`.

1. Les lignes de `local` sont-elles contiguës ? Et celles de `dyn` ?

2. Rangez chaque adresse dans un segment : **code**, **données** (globales), **tas**, **pile**. Les adresses de la pile sont-elles plus grandes ou plus petites que celles du tas ?
3. Faites une pause dans le programme (`getchar()`), affichez son PID avec `getpid()` et, dans un autre terminal, consultez `cat /proc/<pid>/maps` : retrouvez les zones `[heap]` et `[stack]`.

> **PIÈGE CLASSIQUE** **getpid et -std=c17 : la convention du module**
>
> `getpid()` n'appartient pas au C standard mais à **POSIX**. En compilant en `-std=c17` strict, certaines fonctions POSIX ne sont pas déclarées par les en-têtes (avertissement « implicit declaration »). La convention du module : écrire **en toute première ligne** du fichier, avant les `#include` : `#define _POSIX_C_SOURCE 200809L` puis `#include <unistd.h>` pour `getpid`. Nous l'appliquerons dans tous les programmes système des sujets suivants.

> **CULTURE** **La mémoire d'un processus**
>
Code (text)	Les instructions du programme, en lecture seule, partageables entre processus.
Données	Variables globales et `static` : initialisées (data) ou mises à zéro (bss).
Tas (heap)	Blocs de `malloc`/`calloc`/`realloc` ; grandit vers les adresses hautes.
Pile (stack)	Variables locales, paramètres, adresses de retour ; grandit vers le bas.

Drapeau : `FLAG{` + l'écart en octets entre `local[0]` et `local[1]` + `}`.

Mission 3 — La chasse aux fantômes

Les erreurs de gestion de la mémoire sont la première source de failles de sécurité dans les programmes C : selon Microsoft et Google, environ 70 % des vulnérabilités graves de leurs produits sont des erreurs mémoire. Deux outils les débusquent :

- **valgrind** exécute le programme dans une machine virtuelle qui surveille chaque accès : `valgrind --leak-check=full ./prog`.
- **AddressSanitizer** (ASan) instrumente le programme à la compilation (`-fsanitize=address -g`) : plus rapide, il arrête le programme à la première erreur avec un rapport précis.

EX 5 Les quatre fantômes ★★ 20 min

Récupérez `code/sujet04/fantomes.c`. Le programme cache **quatre fantômes**, un par numéro passé en argument (`./fantomes 1` à `./fantomes 4`).

1. Compilez **sans** ASan et lancez chaque numéro : le programme semble-t-il fonctionner ? (Lisez au passage les avertissements `-Wuse-after-free` : ils sont voulus.)
2. Lancez chaque numéro sous `valgrind --leak-check=full`, puis compilez avec `-fsanitize=address` et relancez. Pour chaque fantôme, notez le **nom de l'erreur** et la **ligne** indiqués.
3. Corrigez les quatre fantômes, puis vérifiez que valgrind affiche `All heap blocks were freed -- no leaks are possible` et `ERROR SUMMARY: 0 errors`.

Fantôme	Nom	Danger
1	Fuite mémoire	Un serveur qui fuit quelques octets par requête finit par épuiser la mémoire (déni de service).
2	Use-after-free	Le bloc libéré peut être réattribué : lire ou écrire dedans touche les données de quelqu'un d'autre. Grand classique des exploits de navigateurs.

Fantôme	Nom	Danger
3	Double free	Corrompt les structures internes de l'allocateur ; exploitable pour écrire à une adresse choisie.
4	Débordement du tas	Écrase les métadonnées de l'allocateur ou le bloc voisin.

SÉCURITÉ **Les bons réflexes (guide ANSSI)**

- Tester **chaque** retour de `malloc`, `calloc`, `realloc`.
- Après `free(p);`, écrire `p = NULL;` : un second `free(p)` devient inoffensif et un accès plante franchement au lieu de corrompre en silence.
- Un seul propriétaire par bloc : savoir à tout moment **qui** doit le libérer (le documenter dans le commentaire de la fonction).
- Ne jamais libérer un pointeur qui ne vient pas de `malloc` (adresse d'une variable locale, milieu d'un bloc).

Drapeau : `FLAG{` + le nom de l'erreur signalée par ASan pour le fantôme 2 + `}`.

Mission 4 — L'extension : realloc

`realloc(p, nouvelle_taille)` agrandit (ou réduit) le bloc pointé par `p`. Si la place manque derrière le bloc, `realloc` en alloue un nouveau, **copie** les données, libère l'ancien et renvoie la **nouvelle** adresse. En cas d'échec, il renvoie `NULL` et **l'ancien bloc reste intact**.

```c
int *tmp = realloc(tab, nouvelleCapacite * sizeof *tab);
if (tmp == NULL) {
    /* tab est toujours valide : on peut le libérer ou continuer avec */
} else {
    tab = tmp;
}
```

PIÈGE CLASSIQUE **p = realloc(p, ...)**

Écrire `tab = realloc(tab, ...)` est une erreur classique : en cas d'échec, `tab` reçoit `NULL` et l'adresse de l'ancien bloc est **perdue** (fuite). Passez toujours par un pointeur temporaire. Autre piège : après `realloc`, tous les autres pointeurs vers l'ancien bloc sont invalides.

EX 6 Le tableau extensible

★★ 15 min

Écrivez une structure `Vecteur` (`int *donnees; size_t taille; size_t capacite;`) et une fonction `int ajouter(Vecteur *v, int x)` qui ajoute un élément et **double** la capacité quand le tableau est plein (capacité initiale 4). Le programme lit des entiers jusqu'à la fin de fichier (Ctrl+D), affiche après chaque ajout la taille, la capacité et l'adresse du bloc, puis la somme.

1. L'adresse du bloc change-t-elle à chaque agrandissement ?
2. Pourquoi doubler la capacité plutôt que l'augmenter de 1 ? (Comptez les copies effectuées pour 1 000 ajouts dans chaque cas.)
3. Testez sous valgrind : 0 fuite.

Drapeau : `FLAG{` + la capacité du vecteur après 100 ajouts + `}`.

Mission 5 — Le banc d'essai des tris

Le LAMAE veut comparer des algorithmes de tri sur des tableaux dont la taille est choisie à l'exécution. Chaque algorithme reçoit une **copie** allouée dynamiquement du tableau et remplit une structure de statistiques. Les algorithmes sont rangés dans un **tableau de pointeurs de fonctions** (sujet 03).

Algorithme	Principe	Complexité
Tri à bulles	Compare les éléments voisins et les échange s'ils sont mal ordonnés ; recommence jusqu'à ce que tout soit trié.	$O(n^2)$
Tri par sélection	Cherche le minimum du reste du tableau et le place en tête ; recommence sur le reste.	$O(n^2)$
Tri par insertion	Insère chaque élément à sa place dans la partie déjà triée.	$O(n^2)$, efficace sur des données presque triées
Tri rapide	Diviser pour régner : choisit un pivot et partitionne autour de lui.	O(n log n) en moyenne

Récupérez `code/sujet04/tri_modulaire_depart.c` : les types sont déjà déclarés.

```c
typedef struct {
    long comparaisons;
    long echanges;
    double temps_ms;
} Statistiques;

typedef void (*AlgoTri)(int *tab, size_t n, Statistiques *stats);
```

EX 7 — Le tri à bulles qui compte

★ 10 min

Complétez `triBulle` pour compter les **comparaisons** (`tab[j] > tab[j + 1]`) et les **échanges**. Testez avec `{5, 2, 8, 1, 9}` et affichez les statistiques.

```c
void triBulle(int *tab, size_t n, Statistiques *stats)
{
    stats->comparaisons = 0;
    stats->echanges = 0;
    for (size_t i = 0; i + 1 < n; i++)
        for (size_t j = 0; j + 1 < n - i; j++) {
            /* comparer tab[j] et tab[j + 1], échanger si besoin, compter */
        }
}
```

> **PIÈGE CLASSIQUE** **size_t et n - 1**
>
> Avec `size_t n` (non signé), `n - 1` vaut... 18 446 744 073 709 551 615 si `n == 0`. D'où la condition `i + 1 < n` plutôt que `i < n - 1`.

Drapeau : `FLAG{` + comparaisons + `-` + échanges pour `{5, 2, 8, 1, 9}` + `}` (par exemple `FLAG{12-3}`).

EX 8 — Sélection et insertion

★★ 15 min

Implémentez `triSelection` et `triInsertion` avec leurs statistiques (pour l'insertion, comptez un « échange » par décalage).

```
Tri par sélection
Pour i de 0 à n-2 :
    min = i
    Pour j de i+1 à n-1 :
        Si tab[j] < tab[min] : min = j
    Échanger tab[i] et tab[min]
```

```
Tri par insertion
Pour i de 1 à n-1 :
    clé = tab[i] ; j = i
    Tant que j > 0 ET tab[j-1] > clé :
        tab[j] = tab[j-1] ; j = j - 1
    tab[j] = clé
```

> **INDICE** **Insertion avec un indice non signé**
>
> L'algorithme classique écrit `j = i - 1` et teste `j >= 0` : avec un `size_t`, ce test est **toujours vrai** (boucle infinie et accès hors du tableau). La version ci-dessus, avec `j > 0` et `tab[j-1]`, évite le problème.

EX 9 Les utilitaires dynamiques

★ 10 min

Complétez :

- `int *dupliquerTableau(const int *source, size_t n)` : renvoie une **copie allouée** (à libérer par l'appelant), `NULL` en cas d'échec ;
- `int estTrie(const int *tab, size_t n)` ;
- `int *genererAleatoire(size_t n, int max)` : alloue et remplit `n` entiers dans `[0, max[`.

Le commentaire de chaque fonction doit dire **qui libère** la mémoire renvoyée.

EX 10 Le système de tri modulaire

★★ 20 min

Dans `main`, rangez les algorithmes et leurs noms dans deux tableaux parallèles :

```c
AlgoTri algorithmes[] = { triBulle, triSelection, triInsertion };
const char *noms[] = { "Tri à bulles", "Tri par sélection", "Tri par insertion" };
const size_t nbAlgos = sizeof algorithmes / sizeof algorithmes[0];
```

Puis écrivez le menu :

```
=== SYSTÈME DE TRI MODULAIRE ===
1. Trier avec un algorithme
2. Comparer tous les algorithmes
3. Générer un nouveau tableau
4. Afficher le tableau actuel
5. Quitter
```

- **Option 1** : sous-menu de choix de l'algorithme ; trie une copie, affiche avant/après et les statistiques, libère la copie.
- **Option 2** : pour chaque algorithme, trie une copie et affiche une ligne d'un tableau comparatif (`printf("%-22s %14ld %14ld\n", ...)`), en vérifiant avec `estTrie`.
- **Option 3** : demande une taille, génère un nouveau tableau aléatoire et libère l'ancien.
- Tableau initial : `{64, 34, 25, 12, 22, 11, 90}`.

SÉCURITÉ **Le menu ne doit pas sauter n'importe où**

`algorithmes[choix - 1](copie, n, &s)` avec un `choix` non vérifié lit un pointeur de fonction **hors du tableau** et saute à une adresse arbitraire : c'est une prise de contrôle du programme offerte à l'attaquant. Validez : retour de la saisie, `1 <= choix <= nbAlgos`. Même vigilance pour la taille de l'option 3 (positive, bornée, test du `malloc`). Enfin, vérifiez sous valgrind qu'aucune copie n'est oubliée : 0 octet perdu après une session complète.

POUR LES RAPIDES **Ex 11 — Le grand banc ★★★**

1. **Tri rapide itératif** : implémentez le tri rapide **sans récursion**, avec une pile de bornes simulée par un tableau. L'énoncé d'origine utilisait un tableau de longueur variable `int pile[n];` (VLA) : pour `n = 1 000 000`, cela demande des mégaoctets sur la **pile** (8 Mio par défaut, `ulimit -s`) et plante. Allouez la pile sur le **tas** avec `malloc` (le guide ANSSI déconseille les VLA).
2. **Temps** : mesurez chaque tri avec `clock()` (`<time.h>`) et ajoutez une colonne « Temps (ms) ».
3. **qsort** : ajoutez la fonction `qsort(3)` de la bibliothèque standard, avec un comparateur `int compareInt(const void *a, const void *b)` qui renvoie un nombre négatif, nul ou positif. Pourquoi `return *(const int *)a - *(const int *)b;` est-il faux pour de grands entiers ? Écrivez `return (x > y) - (x < y);`.
4. **Ordre au choix** : un `typedef int (*Comparateur)(int, int);` et un tri à bulles générique qui accepte `croissant` ou `decroissant`.
5. **Grands tableaux** : comparez sur 10, 100, 1 000 et 10 000 éléments, sur un tableau déjà trié et trié à l'envers. Qui gagne sur un tableau presque trié ?

Bilan de la séance

À RETENIR

- `malloc` renvoie un `void *` (pas de cast en C) ou `NULL` : **on teste toujours**.
- La taille se calcule avec `sizeof *p` ; un produit `n * sizeof` se vérifie (`SIZE_MAX`, `calloc`).
- Un `free` par allocation, au bon moment, puis `p = NULL`.
- `realloc` passe par un pointeur temporaire.
- Un tableau 2D dynamique = un tableau de pointeurs + une allocation par ligne ; on libère les lignes avant le tableau.
- valgrind et `-fsanitize=address` font partie du cycle de développement, pas du dépannage d'urgence.

- ☐ J'ai mes 5 drapeaux dans `badge04.txt`.
- ☐ Mes programmes passent sous valgrind avec 0 erreur et 0 fuite.

Aller plus loin

Ressources

Ressource	Pour quoi faire
`man 3 malloc`	`malloc`, `calloc`, `realloc`, `free` et `reallocarray` : valeurs de retour, cas d'erreur.
`man 3 qsort`, `man 3 bsearch`	Tri et recherche génériques de la bibliothèque C.
`man 5 proc` (section `/proc/pid/maps`)	Lire la carte mémoire d'un processus.

Ressource	Pour quoi faire
cppreference — gestion mémoire	Référence normative de `malloc`, `calloc`, `realloc`, `free`, `aligned_alloc`.
Valgrind — Quick Start	Lire un rapport de valgrind.
AddressSanitizer	Les types d'erreurs détectés par ASan.
M. Kerrisk, *The Linux Programming Interface*, ch. 6 et 7	Mémoire d'un processus, allocation sur le tas (`brk`, `sbrk`, `mmap`).
A. Tanenbaum, *Systèmes d'exploitation*	Gestion de la mémoire, mémoire virtuelle.
J.-M. Rifflet, *La programmation sous UNIX*	Le point de vue du programmeur système.
Guide ANSSI (dossier du CM), section allocation dynamique	Règles : retours testés, `free` unique, pointeur remis à `NULL`, pas de VLA.
`ltrace ./prog`	Voir passer chaque appel à `malloc` et `free` (et leurs arguments).
`gdb`	`watch *p` arrête le programme quand la valeur pointée change.

Pour approfondir

CULTURE **Que fait vraiment malloc ?**

`malloc` n'est pas un appel système : c'est une fonction de la `libc` qui découpe de grandes zones demandées au noyau. Pour les petits blocs, elle agrandit le tas avec l'appel système `brk` ; pour les gros (128 Kio par défaut), elle demande une projection mémoire avec `mmap`. Observez-le : `strace -e trace=brk,mmap ./prog`. Chaque bloc est précédé de quelques octets de métadonnées (taille, état) : c'est précisément ce qu'un débordement du tas ou un double free vient corrompre.

CULTURE **Pourquoi malloc ne renvoie presque jamais NULL sous Linux**

Linux pratique l'**overcommit** : `malloc(10 Gio)` peut réussir, la mémoire n'étant réellement fournie qu'au premier accès à chaque page. Si la mémoire vient vraiment à manquer, c'est le *OOM killer* du noyau qui tue un processus. Ce n'est pas une raison pour ne pas tester `NULL` : la norme l'exige, d'autres systèmes (embarqués, Windows) le renvoient, et une limite `ulimit -v` le provoque facilement. Essayez `ulimit -v 100000` dans un sous-shell, puis un gros `malloc`.

CULTURE **Après le C : la mémoire sans free**

Le ramasse-miettes (Java, Python, Go) libère automatiquement les blocs devenus inaccessibles ; Rust vérifie à la compilation qu'un bloc a un seul propriétaire et qu'il est libéré exactement une fois. Les deux suppriment les fantômes de ce sujet, au prix d'une exécution plus coûteuse (ramasse-miettes) ou de règles plus strictes (Rust). C'est l'une des raisons pour lesquelles l'ANSSI, la NSA et la CISA recommandent les langages « memory-safe » pour les nouveaux projets.